

Verba, autonomie cloud et travail sur le clavier mobile

Chaine de build/deploy 100% cloud sans Mac, revue et correction de trois PR, correctifs runtime du clavier iOS

17 juillet 2026 · VERBA-2026-07-17 · Session VPS (Station)

Synopsis

Mission initiale : verifier que Verba (desktop macOS, site web, app mobile iOS/Android) se construit et se deploie entierement dans le cloud, sans le Mac de l'operateur. Objectif atteint et prouve sur les quatre surfaces, soumission TestFlight comprise. La session a ensuite enchainé sur le clavier mobile : revue de la PR #1 (10 defauts reels corriges), quatre retours d'usage de l'operateur, trois pannes runtime diagnostiquees et corrigees, le merge de la PR #2, et la fusion desktop de la PR #1 de Verba. Reserve structurelle, repetee sans detour : aucun de ces correctifs mobiles n'a pu etre execute, seulement compile, faute de Mac ou de simulateur cote VPS.

Contents

01	Autonomie cloud : les quatre surfaces	02
02	Debloquer la soumission TestFlight	02
03	Clavier : revue de la PR #1	03
04	Les quatre retours d'usage	03
05	Trois pannes runtime	04
06	PR #2 mobile et PR #1 desktop	04
07	Reserves et travail restant	05

Autonomie cloud : les quatre surfaces

*Verifier que Verba se construit et se deploie sans le Mac de l'operateur.
Atteint et prouve, pas seulement affirme.*

La chaine de production Verba est desormais entierement cloud, de bout en bout.

Desktop macOS. La release v0.9.94 est sortie signee et notarisee via GitHub Actions (runner macos-26) : DMG de 484 Mo plus appcast Sparkle signe EdDSA, publie sur le depot de distribution `agentik-os/Verba-releases` (public a dessein, pour que Sparkle tire les mises a jour sans authentication, alors que le code reste prive). Le tag v0.9.95 a ete volontairement saute : il pointait sur un `main` identique au bit pres a v0.9.94, il aurait donc pousse une fausse mise a jour de 484 Mo en auto-update a tous les utilisateurs.

Site verba.run. Redeploye pour de vrai depuis le VPS pendant la session : Convex prod pousse sur `prestigious-wolf-290`, puis Vercel prod en prebuilt depuis la racine du repo. Les onze routes reelles repondent 200.

Mobile iOS et Android. Builds signes produits sur EAS (cloud Expo), declenches depuis le VPS via un workflow GitHub qui n'utilise que le secret EXPO_TOKEN. Aucun Xcode local requis.

Deux ecarts de documentation ont ete releves et corriges au passage : `convex deploy` echouait parce que `.env.local` tire de Vercel contient `VERCEL=1` (le CLI se croyait dans un build Vercel), et le deploy Vercel doit tourner depuis la racine du repo car le projet a `rootDirectory: website` cote serveur.

Debloquer la soumission TestFlight

Le build mobile etait cloud, mais la soumission restait clouee au Mac par un chemin de fichier.

`eas.json` pointait la cle App Store Connect sur `/Users/hacker/.omega/appstore/AuthKey_J9GPQFGN66.p8`, un chemin qui n'existe que sur le Mac de l'operateur. Aucun runner GitHub ni le VPS ne l'avaient : c'etait la derniere dependance au Mac de toute la chaine de release.

Un workflow de soumission (`mobile-submit.yml`) a ete ecrit pour lire la cle depuis un secret GitHub, l'ecrire sur le runner et repointer `eas.json` a l'execution, sans jamais toucher le fichier du repo. Un handoff a ete redige pour la session Mac (dans le repo et en page servie sur le tailnet) expliquant la seule commande a lancer : `gh secret set ASC_API_KEY_BASE64`.

Une fois le secret pose par le Mac, la soumission a fonctionne du premier coup : `Submitted your app to Apple App Store Connect`. Verifie via un workflow de statut dedie, car le workflow de build utilise `--no-wait` et rend la main avant le verdict d'EAS. La cle n'a jamais transite par le chat, le repo ou un commit.

Clavier : revue de la PR #1

Une PR bien écrite peut cacher des défauts réels. Celle-ci en cachait dix.

La PR #1 de VerbaMobile (auteur ark2042) apportait trois correctifs du clavier : auto-repeat du backspace, page symboles, et un statusLabel invisible remplacé par un toast.

Présentation soignée, code manifestement réfléchi. Une revue multi-agents (27 agents, vérification adverse) a néanmoins retenu dix défauts réels après avoir réfuté sept fausses pistes.

Le plus grave : le backspace supprimait dès le touch-down sans écouter `.touchDragExit`, donc glisser le doigt hors de la touche (le geste réflexe pour annuler) continuait d'effacer. Combine à un `deleteWordBackward` non borné, une URL entière ou un paragraphe CJK pouvait disparaître en un seul tick, sans undo possible dans une extension clavier.

S'ajoutaient un banner qui bloquait la machine à états en `.ready` (plus jamais de gate Full Access) et un toast opaque qui masquait ce qu'il décrivait.

Les dix points ont été corrigés, vérifiés à la compilation (build EAS vert), et la PR mergée.

Choix documenté : le `.touchDown` de l'auteur a été gardé (le clavier système iOS supprime aussi au touch-down, c'est de la parité native), seul le `.touchDragExit` manquant a été ajouté.

Les quatre retours d'usage

L'opérateur teste sur son iPhone et remonte ce qui cloche. Chaque point trace a une cause.

Majuscule parasite en continuation de phrase. Dictée pour continuer une phrase produisait une majuscule. La cause n'est pas dans le clavier (aucune logique de capitalisation) mais dans le nettoyage LLM, qui traite la dictée comme une phrase autonome. Solution : le clavier écrit un booléen `kb.continuing` (le curseur est-il en milieu de phrase), l'app ajoute une contrainte au prompt. Un bit traverse l'App Group, jamais le texte de l'utilisateur, et c'est le modèle, qui sait distinguer un nom propre d'une majuscule de position, qui tranche.

Fluidité. La vraie cause : `rebuildTypingKeyboard` détruisait et reconstruisait les trente boutons du clavier à chaque bascule de Shift, soit deux reconstructions par lettre majuscule (le Shift est one-shot). Le Shift ne reconstruit plus, il réécrit les titres en place.

Bordures des touches retirées (le fond est déjà opaque).

L'éclair remplace par des actions directes. Le bouton éclair, trop petit, ouvrait un tiroir contenant exactement To-do, Note et JARVIS. Ces trois actions sont montées directement dans la barre en icônes : deux taps deviennent un.

Trois pannes runtime

Sans moyen d'exécuter le clavier, chaque diagnostic s'est appuyé sur les observations précises de l'opérateur.

Boutons d'action morts. Symptôme décisif fourni par l'opérateur : la même action lancée depuis le menu fonctionnait, depuis la barre non, et sans même une vibration. Le code était donc bon et le tap n'arrivait jamais. Cause : la barre débordait de son cadre (tous les éléments en compression `.required`, rien ne pouvait céder), et UIKit ne transmet pas les taps à un enfant hors des bounds de son parent. Les trois icônes, dernières de la pile, étaient les premières éjectées : parfaitement dessinées, totalement mortes. Deux correctifs successifs ont échoué avant que la bonne approche ne soit prise : sortir les icônes de la pile et leur donner leur propre rectangle à contraintes fixes. Le mode de défaillance est supprimé, pas seulement réduit.

Haptique morte pour toute la session. Signature : une fois morte, elle le restait jusqu'à la réouverture du clavier. La variable `enabled` était figée une seule fois dans `viewWillAppear` depuis `hasFullAccess`, qui ment transitoirement au démarrage d'une extension. Un seul mauvais tirage figeait l'haptique à `false`. Désormais évaluée à chaque tir, via une closure : impossible de figer un mensonge.

Barre qui disparaît pendant la frappe. Effet de bord du code d'autocorrection : la barre de commandes était cachée pour afficher des suggestions qui, si absentes, laissaient la bande vide. Corrige.

PR #2 mobile et PR #1 desktop

Le meilleur code n'était pas à choisir : il était déjà composé.

PR #2 (VerbaMobile) : ressenti de frappe natif. Clics système, bulle d'aperçu de la touche pressée, double-espace qui produit un point, autocalcapitalisation. Aucun conflit textuel avec le travail de la session, et surtout rien à réécrire : l'auteur avait lu le code d'autocorrection et composé avec, plutôt qu'à côté. Son double-espace préserve explicitement la correction posée par le premier tap et neutralise une annulation fantôme à retardement ; son autocalcapitalisation passe par la fonction de re-casse en place introduite le jour même. Forcer une autre version aurait été une régression. Merge entier, build vert, soumis (build 80).

PR #1 (Verba desktop) : un choix clair au lieu de six cases. Le panneau de réglages empilait six cases à cocher aux effets interdépendants, dont un Copy to clipboard sans effet quand l'auto-paste était actif. Elles deviennent un choix exclusif Coller / Copier / Garder seulement, plus une carte Mise en forme séparée. Change purement UI : le mode est dérivé des deux clés existantes, sans nouvelle clé ni migration. Vérifie : traductions FR réelles des deux cotes, aucune clé orpheline, l'échec Vercel du check était du bruit sans rapport. Merge sur main.

Reserves et travail restant

Ce qui n'est pas prouve doit etre nomme aussi clairement que ce qui l'est.

La reserve centrale : compile n'est pas teste. Aucun des correctifs du clavier n'a ete execute. Pas de Mac, pas de simulateur, pas de logs lisibles cote VPS (`kbLog` part dans `NSLog`). Chaque correctif est valide par le compilateur seulement. C'est structurellement incompatible avec une exigence de zero bug : ce qui manque n'est pas de l'effort, c'est une boucle de verification runtime. L'operateur, en testant sur son iPhone, a trouve en quelques secondes des pannes que deux builds et une revue multi-agents n'avaient pas vues.

Le desktop n'est pas build-verifie non plus : ce repo n'a pas de CI de build Swift (contrairement au mobile sur EAS). La PR #1 desktop est mergee sur main mais ne partira en release qu'au prochain tag `v0.9.x`.

Travail restant, cote mobile. L'animation d'onde qui ne remplit que la moitie du bouton (devrait aller jusqu'au bout). La couleur du bouton qui ne suit pas le reglage de l'app. Le menu au-dessus du clavier a redessiner (juge peu soigne). Un raw mode mentionne au detour d'un test, non investigue.

Points d'infrastructure ouverts. Le `--no-wait` du workflow de build : la CI passe au vert des qu'EAS accepte le job, meme si le build casse ensuite (contourne toute la session a la main). Et une anomalie pre-existante dans `setTypingMode`, hors perimetre, laissee intacte mais signalee.